

Experiment # 09

Understanding of Best First Search and Beam Search algorithms

Objective/s:

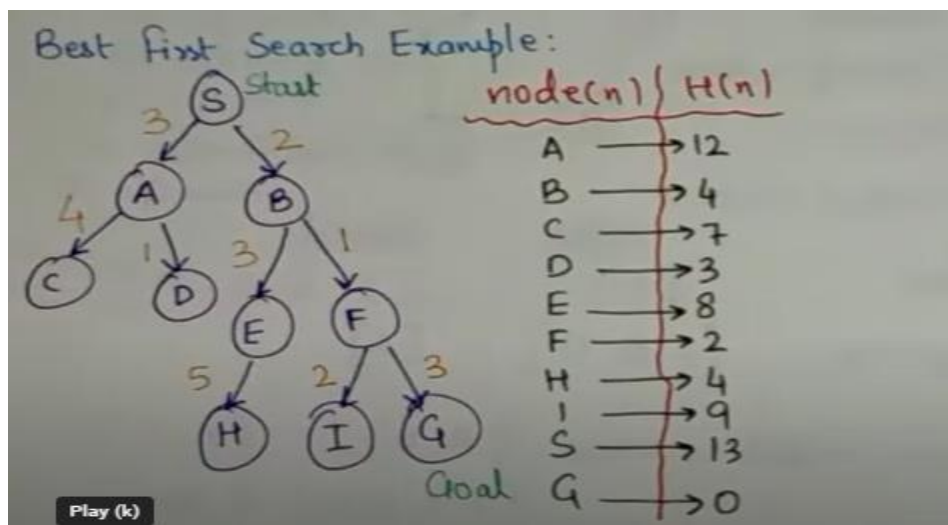
- What is Best First Search and Beam search
- Implementation of BFS and Beam search
- Lab Tasks

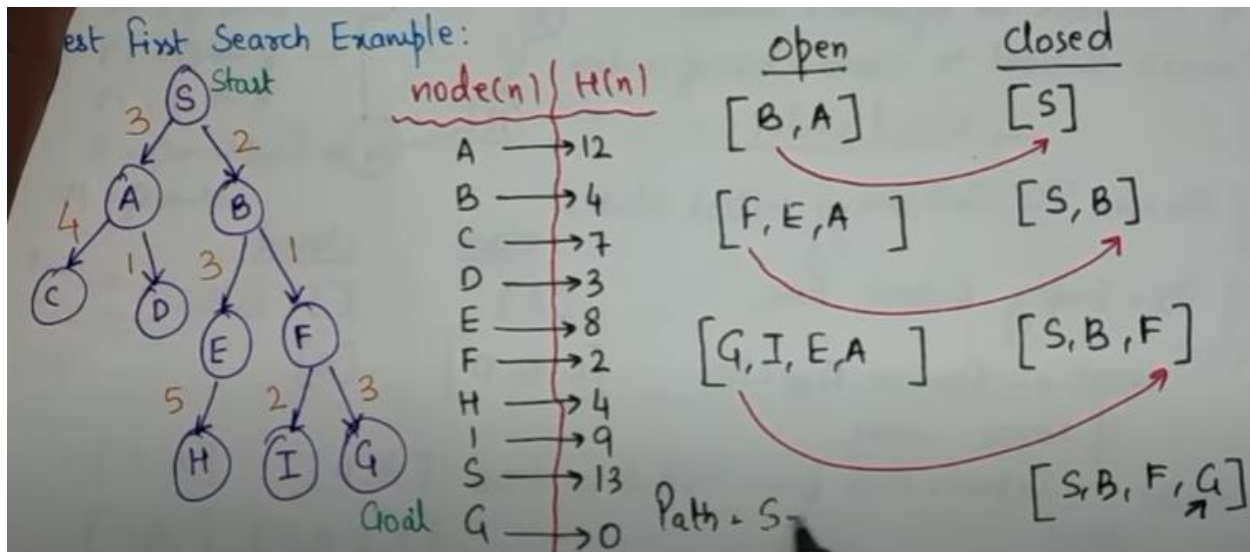
Theory:

Best-first search is a graph traversal algorithm used in computer science and artificial intelligence for finding the shortest path from a starting node to a goal node in a weighted graph. Unlike breadth-first search or depth-first search, which explore all possible paths from the start node, best-first search prioritizes which node to explore next based on a heuristic function that estimates how close a node is to the goal.

At each step, best-first search expands the node that is estimated to be closest to the goal according to the heuristic function. This means it may not explore all possible paths, but it tends to find a solution more efficiently in many cases, especially when the heuristic function provides a good estimate of the distance to the goal.

One common example of best-first search is the A* algorithm, which uses both the actual cost from the start node (g-value) and the estimated cost to the goal node (h-value) to determine the priority of nodes to explore. A* is widely used in pathfinding problems in games, robotics, and other areas of artificial intelligence.





Path = S → B → F → G

Best First Search Algorithm:

The Best First Search algorithm is a general approach to searching in a graph or tree. It is an uninformed search algorithm, meaning it doesn't use any information about the goal or the structure of the search space beyond the provided graph or tree.

Here's a basic outline of the Best First Search algorithm:

1. Initialize a priority queue or a heap with the starting node, with its priority determined by some heuristic function.
2. While the priority queue is not empty:
 - a. Remove the node with the highest priority from the priority queue.
 - b. If the removed node is the goal node, terminate with success.
 - c. Otherwise, expand the removed node by generating its successor nodes.
 - d. For each successor node, calculate its priority based on the heuristic function and add it to the priority queue.
3. If the priority queue becomes empty and the goal node has not been found, terminate with failure.

The effectiveness of the Best First Search algorithm heavily depends on the heuristic function used to calculate the priorities of nodes. The heuristic function estimates the distance from each node to the goal node. A good heuristic function can significantly improve the efficiency of the search by guiding it towards promising areas of the search space.

Advantages:

1. **Efficiency with Good Heuristic Functions:** BFS can be highly efficient when a good heuristic function is used. This function guides the search towards the most promising areas of the search space, potentially reducing the search time significantly.
2. **Space Efficiency:** Unlike algorithms like depth-first search, BFS typically does not require storing the entire search tree in memory, making it more space-efficient, especially for large search spaces.
3. **Optimality with Certain Conditions:** When using an admissible heuristic function (one that never overestimates the cost to reach the goal), BFS guarantees optimality. This means it will always find the shortest path from the start node to the goal node.
4. **Flexibility in Heuristic Functions:** BFS is flexible in terms of the heuristic function used. As long as the heuristic function provides meaningful estimates of the distance to the goal, BFS can be adapted to different problem domains.

Disadvantages:

1. **Dependence on Heuristic Quality:** The effectiveness of BFS heavily depends on the quality of the heuristic function. If the heuristic function is poor or does not provide accurate estimates of the distance to the goal, BFS may perform poorly or even fail to find a solution.
2. **Incomplete in Some Cases:** BFS is not guaranteed to find a solution if an optimal heuristic function is not available. It may get stuck exploring a suboptimal path indefinitely, especially if there are cycles in the search space.
3. **Time Complexity:** While BFS can be efficient with a good heuristic function, it can still have high time complexity in certain scenarios, especially if the search space is large and the heuristic function is not effective in guiding the search.
4. **Memory Requirements for Large Graphs:** Although BFS is generally more space-efficient than some other algorithms like depth-first search, it still requires storing information about explored nodes and their priorities in memory, which can become a limitation for very large graphs.

Example:

```
# Define a simple graph (adjacency list)
graph = {
    'A': ['B', 'C'],
    'B': ['D', 'E'],
    'C': ['F', 'G'],
```

```
'D': [],
'E': [],
'F': [],
'G': []
}

# Define heuristic values for each node (lower means closer to goal)
heuristic = {
    'A': 5,
    'B': 4,
    'C': 3,
    'D': 7,
    'E': 6,
    'F': 2,
    'G': 1
}

def best_first_search(start, goal):
    visited = []
    path = []
    queue = [(start, heuristic[start])]

    while queue:
        # Sort queue based on heuristic value (lowest first)
        queue.sort(key=lambda x: x[1])
        current, _ = queue.pop(0)

        if current in visited:
            continue

        visited.append(current)
        path.append(current)

        if current == goal:
            break

        for neighbor in graph[current]:
            if neighbor not in visited:
                queue.append((neighbor, heuristic[neighbor]))

    return path

# Run best first search from 'A' to 'G'
result_path = best_first_search('A', 'G')
print("Path found by Best First Search:", result_path)
```

Path found by Best First Search: ['A', 'C', 'G']

Beam Search:

Beam search is a heuristic search algorithm commonly used in artificial intelligence and natural language processing, particularly in tasks such as machine translation and speech recognition. It is a variation of best-first search that aims to find the best solution by exploring a limited set of most promising paths at each step, called the "beam width".

Easy Engineering Classes – Free YouTube Lectures
EEC Classes GGSIPU, UPTU, Mumbai Univ., Pune Univ., GTU, Anna Univ., PTU and Others EEC Classes

BEAM SEARCH:
Optimized Version of **Best First Search**. { Only Predetermined no. of Best Partial Solutions are kept as Candidates. }

↳ Heuristic Search Algorithm.
↳ Explores a Graph by expanding the most promising node in a **Limited Set**.
↳ Reduces Memory Requirement.
↳ GREEDY Algorithm.

Beam Value (B) = Predetermined no. of best partial solⁿ are kept as Candidates.

Best-First Search. **Beam-Search** ($B=2$)

Start

Graph with nodes A, B, C, D, E, F, G, H and edge weights:

- A → B: 11
- A → C: 14
- A → D: 7
- B → E: 15
- C → E: 8
- C → F: 10
- D → F: 18
- E → H: 9
- F → G: 20
- H → G: 10

Heuristic values (G):

- A → G = 40
- B → G = 32
- C → G = 25
- D → G = 35
- E → G = 19
- F → G = 17
- H → G = 10
- G → G = 0

Best-First Search tree shows expansion of A to B, C, D, then C to E, F, and F to G.

Beam-Search tree shows expansion of A to B, C, D. Since $B=2$, only the top 2 nodes (B and C) are kept, and D is pruned. Further expansion follows from B and C.

Beam Search Algorithm:

1. **Initialization:** Start with an initial node (or state) representing the start of the search space.
2. **Expansion:** Expand the initial node to generate its successor nodes (or states), representing possible next steps in the search.
3. **Scoring:** Assign a score to each successor node based on some evaluation function. In natural language processing tasks like machine translation, this score may represent the likelihood of a sequence of words being a correct translation or transcription.
4. **Beam Selection:** Select the top-k successor nodes with the highest scores, where k is the beam width. These nodes become the candidates for further exploration in the next step.
5. **Termination:** If any of the selected nodes represent the goal state or if a certain

termination condition is met, terminate the search and return the best solution found so far.

6. **Iteration:** If the termination condition is not met, repeat the process by expanding the selected nodes, scoring their successors, and selecting the top-k nodes for further exploration.

Beam search differs from traditional best-first search in that it maintains only a limited number of paths (controlled by the beam width) instead of all possible paths. This limits the memory requirements and computational complexity of the search, making it more feasible for large search spaces.

Advantages:

1. **Efficiency:** Beam search is more efficient than exhaustive search algorithms like breadth-first search or depth-first search because it explores a limited set of most promising paths at each step. This can significantly reduce the computational resources required, especially in large search spaces.
2. **Scalability:** Beam search is well-suited for large search spaces or problems with a large number of possible solutions. By limiting the number of paths explored, beam search can efficiently handle complex problems that would be impractical to solve exhaustively.
3. **Flexibility:** Beam search is flexible and can be applied to a wide range of problems in artificial intelligence and natural language processing, including machine translation, speech recognition, and sequence generation tasks.

Disadvantages:

1. **Suboptimal Solutions:** Beam search does not guarantee finding the globally optimal solution. Depending on the beam width and the scoring function used, beam search may converge to a suboptimal solution or miss the globally optimal solution altogether.
2. **Beam Width Selection:** The performance of beam search can be sensitive to the choice of beam width. A narrow beam may lead to premature convergence to suboptimal solutions, while a wide beam may increase computational costs and memory requirements.
3. **Scoring Function Design:** The effectiveness of beam search heavily depends on the design of the scoring function used to evaluate candidate solutions. A poorly designed

scoring function may lead to biased or inaccurate evaluations, affecting the quality of the solutions found by beam search.

4. **Limited Exploration:** Beam search may suffer from limited exploration, particularly if the beam width is too narrow or if the search space contains multiple equally promising solutions. This can lead to suboptimal solutions or missed opportunities to find better solutions.

In summary, beam search is a powerful heuristic search algorithm that offers efficiency and scalability for exploring large search spaces. However, it requires careful consideration of parameters such as the beam width and the scoring function to balance exploration and exploitation effectively and to mitigate the risk of converging to suboptimal solutions.

Example

```
# Simple graph (adjacency list)
graph = {
    'A': ['B', 'C', 'D'],
    'B': ['E', 'F'],
    'C': ['G'],
    'D': ['H', 'T'],
    'E': [], 'F': [], 'G': [], 'H': [], 'T': []
}

# Heuristic values (lower is better)
heuristic = {
    'A': 5, 'B': 4, 'C': 3, 'D': 6,
    'E': 7, 'F': 2, 'G': 1, 'H': 5, 'T': 4
}

def beam_search(start, goal, beam_width):
    queue = [(start, heuristic[start])]
    path = []

    while queue:
        # Sort current nodes by heuristic
        queue.sort(key=lambda x: x[1])
        # Keep only the best 'beam_width' nodes
        queue = queue[:beam_width]

        next_queue = []
        for current, _ in queue:
            path.append(current)
            if current == goal:
                return path
            for neighbor in graph[current]:
```

```
        next_queue.append((neighbor, heuristic[neighbor]))  
queue = next_queue
```

```
return path
```

```
# Run beam search with beam width 2  
result = beam_search('A', 'G', beam_width=2)  
print("Beam Search Path:", result)
```

```
Beam Search Path: ['A', 'C', 'B', 'G']
```

Task:

1. Perform Best First Search and beam search algorithm for finding the shortest path in a graph. Assume heuristic values on own.

'A': ['B', 'C', 'D'],

'B': ['E', 'F'],

'C': ['G', 'H'],

'D': ['I', 'J'],

'E': ['K', 'L'],

'F': [],

'G': ['M'],

'H': ['N'],

'I': ['O'],

'J': ['P']